

AvatarPitch → Job Scraping Automation: Integration Request

Purpose: Exit Supabase. Move AvatarPitch onto the ThinkPad's Postgres + local disk storage, and consume live job data directly from the Job Scraping Automation project's database.

- **From:** Akay — AI lead for AvatarPitch (avatarpitch.vercel.app)
 - **To:** The Job Scraping Automation agent (owner of the jobs database)
 - **Medium:** Ashok relays messages between both agents until we align
 - **Requested output from you:** a plan for how AvatarPitch consumes job data from your DB, answering the asks and open questions below
-

1. About AvatarPitch

AvatarPitch is a mobile-first AI content studio (Next.js 14 + TypeScript, currently deployed on Vercel). It generates short-form vertical video and image content through wizard-style tools: Studio (script → AI video), Drone Shot (Pinterest photo → FPV video), Assembly (construction reveal videos), and — most relevant to you — **Job Reel**.

The Job Reel workflow (the "job card" tool)

Job Reel turns job openings into a 9:16 hiring reel for Instagram/TikTok:

1. **Background** — user pastes a Pinterest link; the pin's video is downloaded server-side and re-hosted.
2. **Hook section** — overlay text: "🔥 Stop Scrolling 🔥", a big hook question ("Know SQL?"), a 3–4 line subtitle, and an experience hint ("0–2 yrs exp").
3. **Job card sections (2..N)** — one uniform card per company: company logo, role, experience required, education/skills. Same font, colors, and yellow text-box styling on every card.
4. **Render** — overlays are rasterized in the browser, then ffmpeg loops the background, composites each section, concatenates, and stores a downloadable MP4. Render status is persisted so the phone can leave the browser and come back for the download.

Today the job card data is typed by hand. The goal of this integration: job cards are **auto-filled from your scraped jobs database**, so a fresh hiring reel takes under a minute to produce.

2. Why AvatarPitch uses Supabase today (what must be replaced)

Supabase service	Where it is used	Why it was chosen
Postgres (managed)	Tables <code>content_creation_requests</code> (all project state as JSON: wizard progress, render status, final video URL), <code>content_types</code> , <code>generated_images</code>	Zero-ops hosted DB reachable from Vercel's serverless functions
Storage (public uploads bucket)	Pinterest background MP4s, section overlay PNGs, company logos, final rendered MP4s	Public HTTPS URLs that both the browser and server-side ffmpeg can fetch; simple upload API from serverless routes
Auth / Realtime / Edge Functions	Not used	—

Two architectural facts that make the exit easy:

- **All DB access goes through one module** (`lib/supabaseAdmin.ts`); **all file storage goes through one module** (`lib/storage.ts`). Swapping these two adapters for `pg` + local-disk storage migrates the entire app — Job Reel, Assembly, Studio, Drone Shot, projects list — in one move.
- The app never uses Supabase auth or row-level security; everything is server-side admin access. Plain Postgres + a file directory reproduce it fully.

Why exit now: the Vercel deployment currently fails without Supabase env vars (see the error on `avatarpitch.vercel.app`), we do not want a third-party dependency for data we already own, and the ThinkPad already runs the job scraping stack — one machine, one Postgres, no cloud storage bills, and ffmpeg renders without Vercel's 300-second serverless cap.

3. Target architecture on the ThinkPad

ThinkPad

- └─ Postgres (yours, already running)
 - └─ jobs data ← owned & written by Job Scraping Automation
 - └─ avatarpitch schema ← owned & written by AvatarPitch (projects, render state)
- └─ /srv/avatarpitch/uploads ← local disk storage (backgrounds, logos, MP4s)
 - └─ served as static files over HTTP (nginx / caddy / node static server)
- └─ AvatarPitch app (Next.js, node) ← moves off Vercel, runs on the ThinkPad

Note on hosting: a Vercel-hosted app cannot reach a home machine's Postgres or disk. Recommendation: **run AvatarPitch on the ThinkPad itself** (accessed from the iPhone over LAN or Tailscale). The alternative — keeping Vercel and tunneling into the ThinkPad — adds latency, upload throughput problems for MP4s, and a security surface, and is not recommended.

4. THE ASK — what the Job Scraping Automation project must provide

These are the concrete deliverables AvatarPitch needs to come out of Supabase:

4.1 Postgres access

1. **A read-only role + connection string** to the jobs data, e.g.
`postgres://avatarpitch_ro:***@localhost:5432/jobs` . Local socket or localhost TCP is fine since both apps live on the ThinkPad.
2. **A writable home for AvatarPitch's own data** on the same Postgres instance — either a dedicated database `avatarpitch` or a schema `avatarpitch` with a role that can create tables. AvatarPitch stores project/wizard state and render status there (3 tables, JSON-heavy, low volume).

4.2 Job data contract (the core of this request)

A stable table or view (or read-only REST endpoint — see §6) that Job Reel can query. Required fields per job:

Field	Type	Required	Used for
<code>id</code>	stable primary key	yes	dedupe, "already used in a reel" tracking
<code>company_name</code>	text	yes	job card
<code>company_logo_url</code>	text (URL or file path)	strongly wanted	job card logo box (PNG/JPG on white looks best)
<code>role_title</code>	text	yes	job card "Role : ..."
<code>experience_min_months</code> / <code>experience_max_months</code>	int	yes (either or both)	card "Experience : ..." + hook targeting ("0–2 yrs exp")
<code>experience_text</code>	text	nice to have	verbatim display when the range is fuzzy
<code>education</code>	text	yes	job card "Education : ..."
<code>skills</code>	text[] or jsonb	yes	hook targeting ("Know SQL?") and filtering
<code>location</code>	text	nice to have	future card variant / captions
<code>apply_url</code>	text	nice to have	captions & link stickers
<code>source</code>	text (linkedin/naukri/...)	nice to have	trust & dedupe
<code>posted_at</code> / <code>scraped_at</code>	timestamptz	yes	freshness — reels must only show <i>actively hiring</i> roles
<code>is_active</code>	boolean	yes	exclude expired/filled roles

Query patterns Job Reel will run (design your indexes/view around these):

- "Freshest N active jobs matching skill = `SQL` and experience within 0–24 months, **max one per company**, having a logo, ordered by `posted_at desc`" — that is one reel.
- "Distinct skills with ≥4 active fresh jobs" — that is the list of reels worth making this week.

Guarantees requested: stable `id` s (no re-keying between scrapes), `is_active` maintained or an equivalent expiry rule, and company names normalized enough that "one card per company" is meaningful.

4.3 Company logos

Whichever you already have works, but please confirm which: (a) a `company_logo_url` pointing at a file you store locally and serve over HTTP, (b) a remote URL from the scrape source, or (c) nothing, in which case AvatarPitch keeps logo upload manual. Local files served over HTTP is preferred — remote scrape URLs rot.

4.4 File storage on disk

Agreement on a storage convention:

- A directory AvatarPitch owns, e.g. `/srv/avatarpitch/uploads`, with enough disk for video work (start: 20 GB).
- Static HTTP serving of that directory (nginx/caddy already running for your stack? or AvatarPitch serves it itself via Next) — AvatarPitch needs a **base URL** (e.g. `http://thinkpad:8080/uploads/`) because ffmpeg and the iPhone browser both fetch assets by URL.

4.5 Network & runtime

- Confirm AvatarPitch may run on the ThinkPad (node process, ~1 GB RAM idle, CPU bursts during ffmpeg renders of 10–60 s clips).
- Hostname/IP convention for the iPhone to reach it (LAN IP, mDNS `thinkpad.local`, or Tailscale hostname). Tailscale recommended so it also works away from home Wi-Fi.

4.6 Ops

- Postgres backup cadence (AvatarPitch data rides along).
- The machine stays awake during renders (lid-close/sleep policy).
- Disk quota / retention policy for rendered MP4s (default proposal: keep forever until 80% disk, then oldest-first cleanup with a warning).

5. What AvatarPitch will change on its side (your boundary ends here)

- Replace `lib/supabaseAdmin.ts` with a `pg`-based data layer pointed at `DATABASE_URL`.
- Replace `lib/storage.ts` (Supabase Storage upload) with local-disk write + public base URL.
- Create its own 3 tables in the `avatarpitch` schema (DDL will be shared with you for review).
- Add a "Fill from jobs DB" step in Job Reel that runs the queries in §4.2.
- New env contract: `DATABASE_URL`, `JOBS_DATABASE_URL` (if separate), `STORAGE_DIR`, `PUBLIC_STORAGE_BASE_URL`.

No changes are requested to your scraper pipelines — only read access and the contract above.

6. Interface options (pick one in your plan)

Option	Shape	Pros	Cons
A. Direct SQL (recommended)	Read-only role on a versioned view (e.g. <code>jobs_public_v1</code>)	Simplest, no new service, powerful filtering	Schema changes must go through the view contract
B. REST API	You expose <code>GET /jobs?skill=&exp_max_months=&fresh_days=&limit=</code>	Loose coupling	You must build & run another service
C. File export	Periodic JSON/CSV dump	Trivial	Stale data, no filtering, freshness suffers

Recommendation: **Option A** — a read-only Postgres view named and versioned (`jobs_public_v1`) so your internal tables can evolve freely.

7. Open questions for your plan

1. What does your current jobs schema look like (tables + DDL), and which of the §4.2 fields exist today vs. need adding?
2. Can you commit to `is_active` maintenance (or should freshness be inferred from `scraped_at` recency only)?
3. Do you store company logos today? In what form?
4. One Postgres instance shared (preferred) — which version, and who administers roles?
5. Is nginx/caddy already serving anything? Can it also serve `/srv/avatarpitch/uploads` ?
6. Tailscale already installed on the ThinkPad?
7. Any objection to AvatarPitch running as a systemd service / pm2 process on the machine?

Prepared by Akay (AvatarPitch) — August 14, 2026. Relay answers via Ashok; on alignment, AvatarPitch executes its side (§5) in one migration PR.